



火车票管理系统

移动互联网导论 课程大作业

报告人：郑墨涵

程序功能——基础公共模块

登录与注册：提供用户/管理员身份验证，支持账号重复性校验。

身份识别：根据登录账号的角色（admin 或 user）自动跳转至对应的功能界面。

火车订票系统登录



登录

[没有账号？去注册](#)

程序功能——基础公共模块

登录与注册：提供用户/管理员身份验证，支持账号重复性校验。

身份识别：根据登录账号的角色（admin 或 user）自动跳转至对应的功能界面。

新用户注册



注册

[已有账号？去登录](#)

程序功能——管理端

用户维护：

查看所有注册用户列表
支持按用户名搜索及删除违规账号

后台管理系统

退出登录

用户维护

车票列表

订单列表

搜索用户名

搜索

ID: 1 admin

手机: 13800000000 | 创建: 2023-01-01

admin

删除

ID: 2 test

手机: 13900000000 | 创建: 2023-12-29

user

删除

程序功能——管理端

车票管理：

新增车票：录入完整的车票信息（车次、票价、车站、日期、时间、检票口）。

车票维护：查看、删除现有的车票。

后台管理系统

退出登录

搜索车次/车站

新增车票

T101

00

T101

¥500

北京 -> 重庆

日期: 2025-06-12 12:00

检票口: 12B | 时长: 4小时

删除

取消

保存

K300

¥150

西安 -> 成都

日期: 2024-01-02 08:00

程序功能——管理端

订单监控：

查看全系统的用户下单记录，支持按用户名检索订单

后台管理系统

退出登录

用户维护

车票列表

订单列表

搜索下单用户名

搜索

订单号: ORD1767182195805

用户: test

12345678901

车次: T101 | 北京 -> 重庆

发车: 2025-06-12 12:00 | 检票: 12B

删除订单

程序功能——客户端

首页模块：

广告轮播：展示本地图片广告位。

车票搜索：支持按出发站、终点站、日期筛选车票。



程序功能——客户端

订票模块：

查看车票详情（车次、价格、发车时间、历经时长、检票口），输入联系人信息完成模拟下单。

可以根据出发站筛选车票。

程序功能——客户端

个人中心：

我的订单：查看已购买的车票历史记录。

退出登录：清除当前会话并返回登录页面。



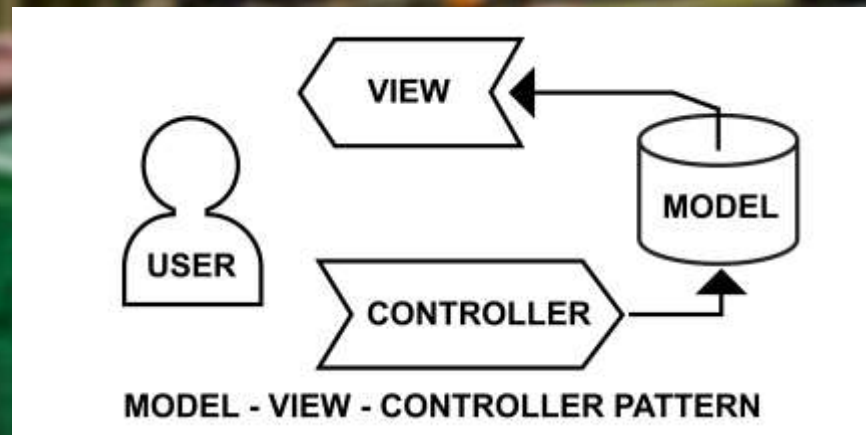
设计思想

- 架构模式：数据-视图分离

- 数据层 (DataManager)：模拟后端，负责所有数据的增删改查逻辑。

定义了User、Train、Order接口，并提供了用户认证与会话管理、车票查询与筛选、订单创建与关联以及管理员功能的方法。

- 视图层 (View/Page)：纯粹负责 UI 展示和调用数据接口。
- 双向同步：利用 ArkTS 的 @State 和 @Prop 装饰器，实现数据变动时界面的自动刷新



设计思想

DataManager 类
static instance: DataManager

唯一的实例（单例）
trains: [G101, D202]
users: [admin, test]
currentUser: null

- 单例模式
- 为了保证数据在不同页面（如登录页跳转到主页）之间能够共享，DataManager 被设计为单例模式。无论在哪个页面调用，操作的都是同一个内存对象（数组），模拟了数据库的持久化特性。

设计思想

- 组件化设计

- 将复杂的界面拆分为独立的组件，提高了代码的可复用性和可维护性。



TrainItemCard: 车票信息展示卡片
(在首页和动车列表复用)



BookDialog: 购票弹窗
(在首页和动车列表复用)

关键技术点——ArkUI 状态管理

@State：用于组件内部的 UI 刷新
(如输入框内容、弹窗开关)。

DataManager (数据源)

getAllUsers/getAllTrains

↓

组件@State变量

用户操作：删除/新增

DataManager (数据源)

refresh()重新获取

↓

组件@State变量

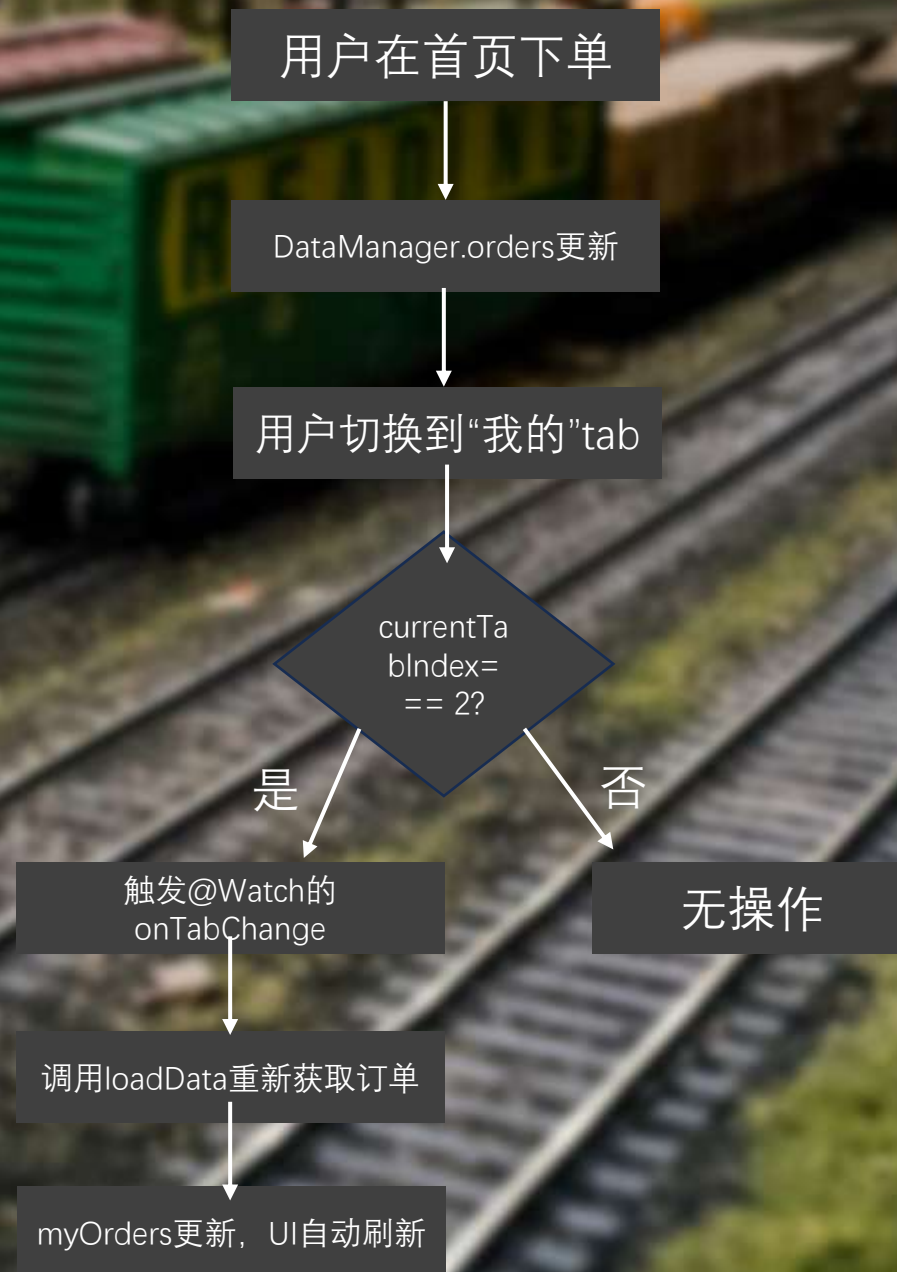
关键技术点——ArkUI 状态管理

- @Watch: 用于监听 Tab 切换。
- 当用户点击“我的”页面时，触发数据重新加载，解决数据缓存导致订单不显示的问题。

```
@Prop @Watch('onTabChange') currentTabIndex: number;
```

作用：监听currentTabIndex的变化。

只有当值实际发生变化时（从1变为2），自动调用onTabChange()方法



关键技术点——ArkUI 状态管理

- @Prop：实现父子组件间的单向数据传递。

父组件: @State

数据可修改，驱动UI更新

传递

子组件: @Prop

数据只读，接收变化

关键技术点——布局技术

Tabs容器：实现管理员顶部和用户底部导航栏结构。



```
Tabs({ barPosition: BarPosition.End }) { // 设置Tabs位置在底部
  TabContent() { HomeView() }           // 定义第一个内容页
    .tabBar(this.TabBuilder("首页", 0, $r('app.media.startIcon'))) // 定义对应的标签栏

  TabContent() { TrainListView() }       // 第二个内容页
    .tabBar(this.TabBuilder("动车", 1, null))

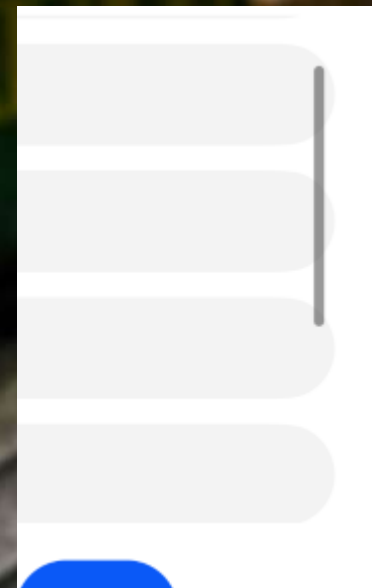
  TabContent() { MineView({ currentTabIndex: this.currentIndex }) } // 第三个内容页
    .tabBar(this.TabBuilder("我的", 2, null))
}
.onChange((index) => { this.currentIndex = index; }) // 监听切换事件
```


关键技术点——布局技术

List 提供滚动容器。

ForEach 基于数据数组生成组件，
可动态渲染车票和订单列表。

```
List() { // 创建可滚动的列表容器
  ForEach(this.resultTrains, //数据源
    (item: Train) => { // 每个数组元素
      ListItem() { // 列表项容器，用于性能优化
        TrainItemCard({ train: item, onBook: () => { ... } }) //自定义卡片组件
      }
    }, (item: Train) => item.trainNo // 关键：生成项的唯一标识符，用于UI更新
  )
}
.layoutWeight(1) // 占满剩余空间
```



关键技术点——布局技术

Swiper: 实现首页广告位的自动轮播展示。



关键属性:

autoplay: 布尔值，控制是否自动轮播。

interval: 自动轮播间隔（毫秒）。

indicator: 是否显示页面位置指示器。

loop: 是否循环播放（默认为 true）。

vertical: 是否垂直轮播（默认为 false 水平）。

关键技术点——布局技术

RelativeContainer/Column/Row:
实现复杂的表单和卡片布局。

```
Column() { // 卡片整体垂直排列
  Row() { // 第一行：车次和价格水平排列
    Text(this.train.trainNo)
    Text('¥${this.train.price}')
  }.width('100%').justifyContent(FlexAlign.SpaceBetween) // 两端对齐

  Row() { // 第二行：车站和时间信息水平排列
    Column() { ... } // 起始站和时间（嵌套Column垂直排列）
    Column() { ... } // 历时和虚线
    Column() { ... } // 终点站和检票口
  }.width('100%').justifyContent(FlexAlign.SpaceAround) // 均匀分布

  Button("预定") // 第三行：按钮
}
```



G101 ¥500

北京 4小时 上海

09:00 ----- 检票:12A

[预定](#)

关键技术点——路由与生命周期

`router.replaceUrl`: 实现登录后的页面跳转, 同时销毁登录页。

`replaceUrl` 会先销毁当前页面, 再将目标页入栈。这防止了物理返回键导致的不合逻辑的回退。

登录成功: `replaceUrl('UserMainPage')` → 页面栈: `[UserMainPage]` ← `Index`被销毁

用户按返回键 → 退出应用 (登录状态单向流转)

再进入Admin: `replaceUrl('AdminPage')` → 页面栈: `[AdminPage]` ← `UserMainPage`被销毁

关键技术点——路由与生命周期

aboutToAppear: 在组件加载前初始化数据, 确保页面显示时已获取到最新的车票信息。

创建阶段: Constructor → aboutToAppear → build → 组件渲染

活跃阶段: 用户交互 → 状态更新(@State) → build重新执行 → UI更新

隐藏/显示阶段: 页面切换 → aboutToDisappear (离开) → aboutToAppear (再次进入)

销毁阶段: 页面被replace或应用退出 → aboutToDisappear → 组件销毁

关键技术点——资源管理

本地资源引用：通过 `$r('app.media.filename')` 统一管理图片资源。

`$r('app.media.banner1')` // 引用资源作为广告位轮播图

`$r('app.media.startIcon')` // 引用资源作为应用启动图

`$r('app.media.startIcon')` // 引用资源作为用户头像

